

SE446: Big Data Engineering

Week 9A: Apache Kafka — Distributed Messaging

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 From Batch to Real-Time
- 2 Kafka Architecture Deep Dive
- 3 Kafka in Practice
- 4 Kafka Design Decisions
- 5 Summary

Outline

- 1 From Batch to Real-Time
- 2 Kafka Architecture Deep Dive
- 3 Kafka in Practice
- 4 Kafka Design Decisions
- 5 Summary

Learning Objectives

By the end of this session you will be able to:

- 1 Explain why batch processing alone is insufficient for modern data systems
- 2 Describe Kafka's architecture: brokers, topics, partitions, offsets
- 3 Understand producers, consumers, and consumer groups
- 4 Publish and consume messages using CLI and Python
- 5 Reason about partitioning strategies and message ordering
- 6 Compare Kafka with traditional message queues

What's New

Until now, all our tools (HDFS, MapReduce, Hive, Spark) assumed **data already exists**. Today we learn how to handle data that **arrives continuously**.

The Big Data Journey — Where Are We?



What We Built So Far

Store (HDFS) → Process (MR/Spark)
→ Query (Hive/SQL) → Optimize + ML

The Missing Piece

All of these assume data **already sits** on disk. What if it **arrives non-stop**?

The Problem: Batch Is Too Late



Fraud Detection

Decision needed in 50ms.
Nightly batch = 12h too late.



Surge Pricing

Demand changes per minute.
Hourly batch = wrong prices.



IoT Sensors

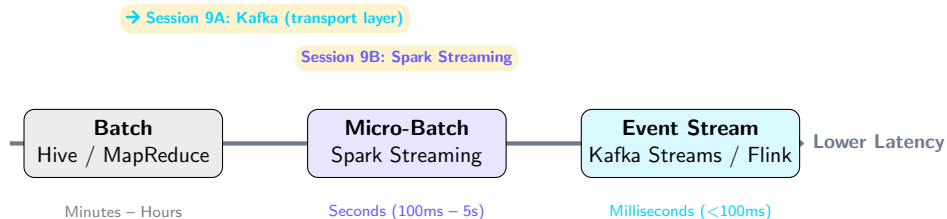
Temperature spike at 2:03 PM
should alert at 2:03 PM.

Key Distinction

Batch processing answers: *"What happened?"*

Real-time processing answers: *"What is happening right now?"*

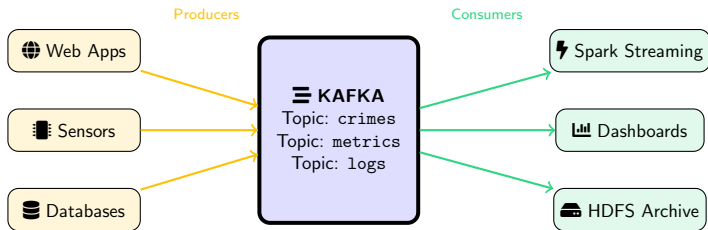
The Paradigm Spectrum



Today's Scope

Kafka = the **data transport layer** (moves data between systems). Session 9B = Spark Structured Streaming (processes the stream).

Where Kafka Fits in the Architecture



The Key Idea: Decoupling

Kafka is a **central hub**. Producers and consumers are **independent** — they don't need to know about each other. Any producer can write; any consumer can read; at any time.

Outline

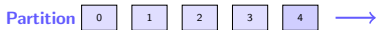
- 1 From Batch to Real-Time
- 2 Kafka Architecture Deep Dive
- 3 Kafka in Practice
- 4 Kafka Design Decisions
- 5 Summary

Core Concepts at a Glance

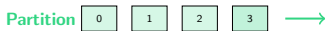
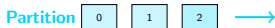
Concept	Analogy	Definition
Broker	Post office	A Kafka server that stores messages and serves reads/writes
Topic	Mailbox category	A named feed of messages (like a database table)
Partition	Sorting bin	A topic is split into partitions for parallelism
Offset	Letter number	Sequential ID of a message within a partition
Producer	Sender	Application that writes messages to a topic
Consumer	Recipient	Application that reads messages from a topic
Consumer Group	Team of recipients	Consumers that share the reading workload

Topics and Partitions

Topic: "crimes" (3 partitions)



Arrow = latest offset
(where next message goes)



- ✓ Within a partition: **ordered, immutable**
- ✗ Across partitions: **no ordering guarantee**
- + More partitions = more parallelism

Design Decision

Partitions are the unit of parallelism. Choose the number of partitions based on your expected consumer count and throughput needs.

How Messages Are Assigned to Partitions

No Key: Round-Robin

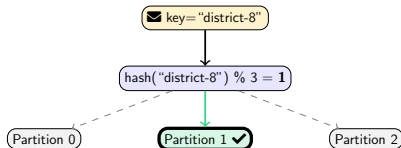
Messages are spread evenly across all partitions.
Good for **load balancing**.

```
# No key -> round-robin
producer.produce("crimes",
    value="theft at district 8")
```

With Key: Hash Partitioning

$\text{hash}(\text{key}) \% \text{num_partitions}$
Same key $\rightarrow$ same partition always.

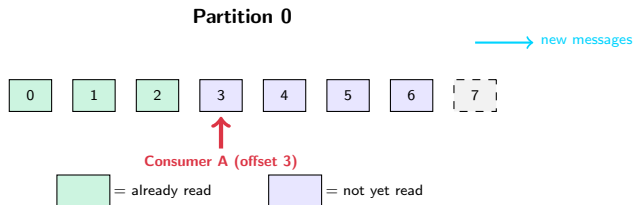
```
# With key -> hash to partition
producer.produce("crimes",
    key="district-8",
    value="theft at district 8")
```



Why Use Keys?

Ordering: All events for the same entity are in order. **Locality:** Related events are co-located for efficient downstream processing.

Offsets — The Consumer's Bookmark



Starting Positions

earliest: Read from offset 0

latest: Read only new messages

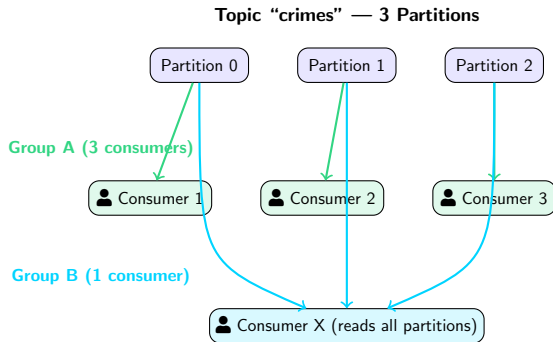
Retention

Kafka **retains** messages (default: 7 days).
Consumers can go back and re-read!

Key Insight

Unlike RabbitMQ where a consumed message is **deleted**, Kafka **keeps messages**. This enables replay, auditing, and multiple independent consumers reading the same data.

Consumer Groups — Scaling Reads



Rule

1 partition → exactly
1 consumer per group

Consumers > Partitions

Extra consumers sit **idle**
(no partition to read)

Two Groups

Each group gets **all**
messages (pub/sub)

Kafka vs Traditional Message Queues

Feature	Apache Kafka	RabbitMQ
Message storage	Retained on disk (configurable TTL)	Deleted after acknowledgement
Message replay	✓ Reset consumer offset	✗ Messages are gone
Throughput	1M+ msgs/sec per broker	~50K msgs/sec
Ordering	Guaranteed per partition	Guaranteed per queue
Multiple readers	Groups independently consume all msgs	Message consumed once
Primary use case	Event streaming, log aggregation	Task queue, RPC

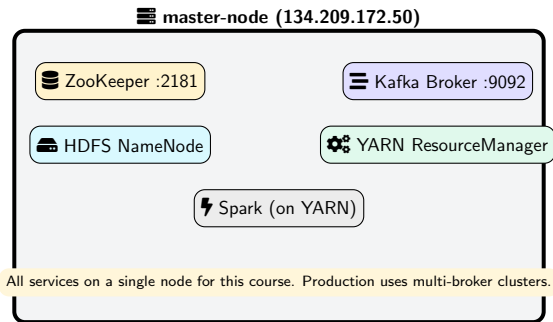
⚠ Important

Kafka is **NOT** a replacement for RabbitMQ. Different tools for different problems. Kafka = event log. RabbitMQ = work queue.

Outline

- 1 From Batch to Real-Time
- 2 Kafka Architecture Deep Dive
- 3 Kafka in Practice
- 4 Kafka Design Decisions
- 5 Summary

Our Cluster Kafka Setup



Connection Info

`bootstrap.servers = 134.209.172.50:9092` — use this in all your producers and consumers.

CLI Demo — Create, Produce, Consume

```
# Step 1: Create a topic with 3 partitions
kafka-topics.sh --create --topic demo-crimes \
  --bootstrap-server localhost:9092 \
  --partitions 3 --replication-factor 1

# Step 2: List all topics
kafka-topics.sh --list --bootstrap-server localhost:9092

# Step 3: Start a consumer (Terminal 1 -- listening)
kafka-console-consumer.sh --topic demo-crimes \
  --bootstrap-server localhost:9092

# Step 4: Produce messages (Terminal 2 -- typing)
kafka-console-producer.sh --topic demo-crimes \
  --bootstrap-server localhost:9092
> {"type":"THEFT","district":8,"hour":14}
> {"type":"BATTERY","district":11,"hour":22}
```

What You'll See

As soon as you type a message in Terminal 2, it appears **instantly** in the consumer in Terminal 1.

Python Producer

```
from confluent_kafka import Producer
import json, time

conf = {'bootstrap.servers': '134.209.172.50:9092'}
producer = Producer(conf)

crimes = [
    {"id": 1, "type": "THEFT", "district": 8, "arrest": False},
    {"id": 2, "type": "BATTERY", "district": 11, "arrest": True},
    {"id": 3, "type": "ROBBERY", "district": 3, "arrest": False},
]

for crime in crimes:
    producer.produce(
        topic="crimes",
        key=str(crime["district"]),      # partition by district
        value=json.dumps(crime).encode("utf-8")
    )
    print(f"Produced: {crime['type']} in district {crime['district']}")
    time.sleep(1)

producer.flush()      # block until all messages are delivered
```

Key Points

`produce()` is **async**. `flush()` blocks until delivery is confirmed. Key = district → all crimes from the same district go to the same partition.

Python Consumer

```
from confluent_kafka import Consumer
import json

conf = {
    'bootstrap.servers': '134.209.172.50:9092',
    'group.id': 'crime-analytics-group',
    'auto.offset.reset': 'earliest'
}

consumer = Consumer(conf)
consumer.subscribe(['crimes'])

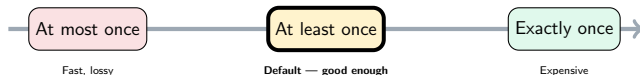
try:
    while True:
        msg = consumer.poll(timeout=1.0)    # wait up to 1 second
        if msg is None:
            continue
        if msg.error():
            print(f"Error: {msg.error()}")
            continue
        crime = json.loads(msg.value().decode("utf-8"))
        print(f"Type: {crime['type']} | Partition {msg.partition()} "
              f"| Offset {msg.offset()}")
finally:
    consumer.close()
```

Three Key Settings

`poll()` = **pull model** (consumer controls pace). `group.id` = consumer group membership.
`auto.offset.reset=earliest` = start from first message.

Delivery Guarantees

Guarantee	Meaning	Kafka Config
At most once	Message may be lost, never duplicated	acks=0
At least once	Message never lost, may be duplicated	acks=1 or acks=all (default)
Exactly once	Neither lost nor duplicated	Idempotent producer + transactions



Practical Advice

Most real systems use **at-least-once** with **idempotent consumers**. Exactly-once semantics are hard and add significant overhead.

Outline

- 1 From Batch to Real-Time
- 2 Kafka Architecture Deep Dive
- 3 Kafka in Practice
- 4 Kafka Design Decisions**
- 5 Summary

How Many Partitions?

Factor	Recommendation
Expected throughput	More partitions = higher throughput (each partition = sequential I/O)
Consumer parallelism	At least as many partitions as max consumer instances in a group
Ordering requirements	Fewer partitions if strict global order matters
Cluster size	Don't exceed 10K partitions per broker



Rule of Thumb

Start with the **number of consumers you expect**, then **double it** for future scaling. You can increase partitions later, but you **cannot decrease** them.

Retention Policy — Kafka Is Not a Queue

```
# Time-based: keep messages for 7 days (default)  
log.retention.hours=168  
  
# Size-based: keep up to 1 GB per partition  
log.retention.bytes=1073741824  
  
# Compaction: keep only latest value per key  
log.cleanup.policy=compact
```

Traditional Queue (RabbitMQ)

Read → Acknowledge → **Delete**
“Fire and forget”

Kafka Log

Read → Acknowledge → **Keep**
“Replayable event log”

Why This Matters

Replayability: A newly deployed consumer can read all historical data. A crashed consumer can restart from its last committed offset — no data loss.

Kafka in the Real World

Company	Use Case	Scale
LinkedIn	Activity tracking, metrics	7 trillion msgs/day
Netflix	Real-time recommendations	1 PB/day
Uber	Trip events, pricing, matching	Millions of msgs/sec
Twitter/X	Stream processing, analytics	Global event bus

Why So Popular?

Kafka combines four properties that no other system offered together:

(1) High throughput + (2) Fault tolerance + (3) Persistence + (4) Decoupling

💡 Fun Fact

Kafka was created at **LinkedIn** in 2011 and named after the author Franz Kafka — because it was designed for **writing** (data).

Outline

- 1 From Batch to Real-Time
- 2 Kafka Architecture Deep Dive
- 3 Kafka in Practice
- 4 Kafka Design Decisions
- 5 Summary

Key Takeaways

- 1 **Why Kafka?** Batch processing is too slow for real-time scenarios. Kafka provides a high-throughput, persistent message transport layer.
- 2 **Architecture:** Broker → Topic → Partition → Offset. Messages are **ordered within a partition**, retained on disk.
- 3 **Producers** write messages (optionally with keys for partitioning). **Consumers** read at their own pace using the pull model.
- 4 **Consumer Groups** split partition ownership for parallel reading. Different groups independently consume all messages (pub/sub).
- 5 **Not a queue:** Messages are **retained**, not deleted after read. This enables replay, multi-consumer patterns, and auditing.



Core Formula

Producers → Kafka (Topics/Partitions) → Consumer Groups

What's Next

Session 9B: Spark Structured Streaming

- The “infinite table” model — process streams with the DataFrame API
- `readStream` from Kafka → parse JSON → aggregate → `writeStream`
- Output modes: append, update, complete
- Time windows: tumbling and sliding
- Watermarks for handling late data

Today's Lab: Kafka Fundamentals

- Create topics, produce crime messages with Python
- Observe partition assignment with and without keys
- Experiment with consumer groups (1, 2, 3 consumers)
- Monitor consumer lag

Milestone M5 Reminder

Due Week 12: End-to-end streaming pipeline (Kafka → Spark Streaming). Today you build the Kafka foundation.